

FOL: A Functional Object Lisp

Frank Adrian
European Lisp Symposium 2026

Abstract—We present FOL (Functional Object Lisp), a new Lisp dialect that combines persistent functional data structures from Clojure with CLOS-style object-oriented programming and Dylan-inspired modules and naming conventions. FOL features a bootstrap implementation in Common Lisp, leveraging the FSet and Sycamore libraries for persistent collections. The language provides Clojure-compatible syntax and semantics for sequence operations and transducers while adding pattern dispatch and maintaining full compatibility with the Common Lisp Object System (CLOS) Meta-Object Protocol (MOP). Key innovations include general predicate specializers in pattern matching and nested predicate support in destructuring patterns. We demonstrate how FOL bridges concepts from multiple Lisp traditions and present a self-hosted meta-circular evaluator that showcases the language’s metaprogramming capabilities.

Index Terms—Functional programming, object-oriented programming, Lisp, persistent data structures, CLOS, MOP, transducers

I. INTRODUCTION

The Lisp family of languages has evolved along multiple paths, each emphasizing different aspects of the language’s core philosophy. Common Lisp [1] provides a mature, standardized platform with powerful object-oriented features through CLOS [2] and metaprogramming through the MOP [3]. Clojure [4] introduced persistent data structures and functional programming idioms to the JVM ecosystem, emphasizing immutability and simplicity. Dylan [5] explored an object-oriented Lisp with a clear type system.

FOL (Functional Object Lisp) synthesizes ideas from these traditions to create a language that:

- Provides Clojure-style persistent data structures with structural sharing
- Maintains CLOS-style generic functions with pattern dispatch
- Adopts Dylan’s naming conventions (`<type>`) for clarity and its module structure which is simpler and more intuitive than Clojure’s namespaces or Common Lisp’s packages
- Supports both functional and object-oriented programming paradigms
- Extends pattern matching with general predicate specializers, enabling dispatch on arbitrary predicates including support for nested predicates in destructuring patterns

This paper describes FOL’s design, implementation, and contributions to the Lisp ecosystem. Our main contributions include: (1) seamless integration of persistent data structures with CLOS and its MOP, (2) general predicate specializers that work with any function, including nested predicates in destructuring patterns, and (3) a self-hosted evaluator demonstrating the language’s metaprogramming capabilities.

II. LANGUAGE DESIGN

A. Core Philosophy

FOL is built on the principle that persistent data structures and object-oriented programming are complementary, not antagonistic. While Clojure eschews traditional OOP in favor of protocols and multi-methods, FOL embraces CLOS’s full power while maintaining Clojure’s functional purity through persistence.

The language makes the following design commitments:

- 1) **Immutability by default:** All objects and collections are persistent
- 2) **Structural sharing:** Updates to objects and collections create new versions sharing structure with old
- 3) **Object identity:** Objects have identity separate from value equality
- 4) **Generic functions:** Multiple dispatch over destructuring patterns for polymorphism
- 5) **Meta-object protocol:** Full introspection and extension capabilities

B. Syntax and Readability

FOL adopts Clojure’s reader syntax for consistency with functional programming conventions. Functions are defined using `defn` (from Clojure), with parameters specified in vector brackets `[...]`. Local bindings use `bind` (similar to Clojure’s `let`). The language provides rich destructuring capabilities for vectors, dictionaries, and other collections.

```
;; Vectors, Maps (dictionaries), Sets, etc.
[1 2 3], {:name "Alice" :age 30}, #{1 2 3 4},
...

;; Function definition with pattern matching
(defn factorial
  ([[n (= 0)]] 1)
  ([[n (= 1)]] 1)
  ([n]
   (* n (factorial (dec n)))))

;; Nested destructuring with :or for defaults
(defn process-config
  ([{:keys [server port]
    :or {port 8080}
    :as config}]
   (bind [{:keys [host timeout]
    :or {timeout 30}} server]
    {:host host
     :port port
     :timeout timeout})))
```

These examples demonstrate FOL’s rich destructuring capabilities, combining pattern matching with equality tests, default

values, and nested destructuring. The use of angle brackets (<type>) for type names, borrowed from Dylan, provides visual distinction between types and values.

C. Module System

FOL adopts a simplified module system inspired by Dylan, providing a more intuitive alternative to Common Lisp's packages or Clojure's namespaces:

```
(module my-app
  (import fol.core :all)
  (import fol.collection [<vector> <dict> <set>])
  (use-module fol.string)
  (export process-data validate-input)

;; Module definitions
(defn process-data [data]
  ...)

(defn validate-input [input]
  ...)
)
```

The module form defines a new module and its dependencies:

- `import` brings specific symbols or all symbols (`:all`) from another module
- `use-module` imports all public symbols from another module (equivalent to `import ... :all`)
- `export` specifies which symbols are visible to other modules

This design reduces the ceremony of Common Lisp's packages while providing more explicit control than Clojure's implicit namespaces.

D. Macros

FOL's macro system extends Clojure's approach with support for multiple destructuring patterns:

```
;; Traditional single-pattern macro
(defmacro when [test & body]
  `(if ~test
      (do ~@body)))

;; Multi-pattern macro - unique to FOL
(defmacro for-loop
  ([var-init test update & body]
   `(loop [~@var-init]
      (if ~test
          (do ~@body
              (recur ~@update))
          nil)))
  ([bindings test & body]
   `(bind ~bindings
      (while ~test
        ~@body))))
```

Like Clojure, FOL uses syntax-quote (```), unquote (`~`), and unquote-splicing (`~@`). Unlike functions, macros **cannot** use predicate specializers in their parameter lists since macros operate on unevaluated forms.

E. Error Handling

FOL provides Clojure-style exception handling with `try` and `catch`:

```
(defn safe-divide [a b]
  (try
    (/ a b)
    (catch <division-by-zero> e
      (println "Cannot divide by zero")
      nil)
    (catch <error> e
      (println "Error:" (error-message e))
      nil)))
```

Exceptions are objects that can be caught by type. The system provides built-in exception types and supports user-defined exception classes. More sophisticated error handling with restarts and conditions is planned for future releases (see Section VIII-F).

F. Type System

All FOL types derive from the class `<persistent-object>`:

```
<persistent-object>
|- <number>
|- <string>
|- <symbol>
|- <collection>
|   |- <vector>
|   |- <list>
|   |- <dict>
|   +- <set>
+- <user-defined-classes>
```

Primitive types wrap their Common Lisp equivalents in a `val` slot, accessed through the generic function `fol-val`. This allows primitives to participate in the persistent object protocol while maintaining compatibility with Common Lisp's numeric tower.

III. ARCHITECTURE AND IMPLEMENTATION

A. Bootstrap Implementation

FOL is implemented in Common Lisp, leveraging the existing Common Lisp facilities (CLOS/MOP) and libraries: FSet [8], Sycamore, Closer-MOP, PPCRE, etc.

The implementation consists of approximately 6,000 lines of Common Lisp code in the files:

TABLE I
FOL IMPLEMENTATION COMPONENTS

File	LOC	Purpose
persistent	250	Base persistent object protocol
classes	180	Primitive type wrappers
collection	890	Persistent collections
seqop	1420	Sequence operations
eval	1580	Evaluator and special forms
standard-names	1680	Standard environment

B. Persistent Object Protocol

FOL extends CLOS with a persistent object protocol. All user-defined classes inherit from `<persistent-object>` and use `pslot-value` for slot access (but as shown below, objects can also use `assoc` to update slots and keyword functions to access them, as is done in Clojure):

```
(defclass* <person> (<persistent-object>)
  ((name :type <string>)
   (age :type <number>)))

(def alice (make <person> :name "Alice" :age
  30))

;; Updates return new instances
(def older-alice
  (assoc alice :age 31))

;; Original unchanged
(:age alice) ; => 30
(:age older-alice) ; => 31
```

FOL's `defclass*` macro extends CLOS's `defclass` to create persistent classes. It ensures all instances inherit from `<persistent-object>` and automatically implements the persistent slot protocol. Like CLOS, it supports slot options including `:type`, `:initarg`, `:initform`, and `:reader/writer` specifications. Unlike standard CLOS, writer functions created via `:writer` return new persistent objects with the updated slot value rather than modifying the object in place, maintaining immutability.

Persistence is achieved through careful management of slot values and structural sharing. When a slot is updated, only the affected slots are copied; others are shared between instances.

C. Collection Implementation

FOL's collections are implemented as thin wrappers around FSet and Sycamore data structures:

```
(defclass* <collection> (<persistent-object>)
  ((items :type fset:collection)))

(defclass* <vector> (<collection>) ())
(defclass* <dict> (<collection>) ())
(defclass* <set> (<collection>) ())
```

This design provides several benefits:

- Structural sharing inherent to FSet
- $O(\log n)$ updates for most operations
- Compatibility with Common Lisp sequence functions
- Extensibility through CLOS

IV. NOVEL FEATURES

A. Persistent Objects With Fully Integrated MOP

All FOL objects derive from a persistent class that ensures objects themselves are persistent and share structure with earlier versions of themselves. This unifies FOL's object paradigm—the language does not have separate persistence models for scalar objects and collections.

1) Implementation Details: Each FOL object is implemented as a persistent hash-dictionary whose keys are slot names and values are slot values. Slot access takes $O(\log n)$ time. In practice, due to the large branching factor in hash-dictionaries (32-64) and the small number of slots in most objects (typically < 20), slot access performs effectively like constant time for typical use cases.

The persistent object protocol extends CLOS with three key operations:

- 1) **pslot-value:** Read a slot value (like CLOS's `slot-value`)
- 2) **set-pslot-value:** Create a new instance with an updated slot value
- 3) **pslot-boundp:** Check if a slot is bound (like CLOS's `slot-boundp`)

When a slot is updated via `set-pslot-value`, the system creates a new object instance sharing the underlying hash-dictionary structure with the original. Only the modified slot's entry is updated; unchanged slots share their storage with the previous version. This provides efficient structural sharing while maintaining immutability.

The MOP integration allows classes to customize persistence behavior through standard CLOS mechanisms. For example, classes can define `initialize-instance` methods, specify slot options (`:type`, `:initform`, `:reader`, `:writer`), and use multiple inheritance—all while maintaining persistence.

This approach contrasts with Clojure's records and protocols, which provide less flexibility, and with standard CLOS, which lacks persistence. FOL demonstrates that CLOS's power and persistence can coexist without compromise.

B. Generic Function Integration

FOL's generic functions fully integrate with persistent data. Methods dispatch on type predicates (e.g., `<vector>?`, `<dict>?`) which check both the specified type and all subtypes, providing flexible polymorphism:

```
(defgeneric process [x])

(defmethod process [(<vector>? x)]
  (map process x))

(defmethod process [(<dict>? x)]
  (update-vals x process))

(defmethod process [(<number>? x)]
  (* x 2))

;; Works recursively on nested structures
(process [1 {:a 2 :b 3} [[4]]])
; => [2 {:a 4 :b 6} [[8]]]

;; Multiple dispatch with type predicates
;; and destructuring
(defgeneric compute-total [items discount])

;; Dispatch on collection type
;; with destructuring
(defmethod compute-total
```

```

[(<vector>? items) (<number>? discount)]
(* (reduce + items) (- 1.0 discount)))

;; Dispatch using both type and predicate
(defmethod compute-total
  [(<dict>? items)
   { :keys [rate type] :or {type :percentage}}]
  (bind [subtotal (reduce + (vals items))]
    (if (= type :percentage)
        (* subtotal (- 1.0 rate))
        (- subtotal rate))))

```

This demonstrates how generic functions provide polymorphism through multiple dispatch while maintaining destructuring capabilities and functional purity.

C. Predicate Specializers

This is one of FOL's main contributions to the Lisp ecosystem.

FOL extends pattern matching with general predicate specializers that allow functions to dispatch based on arbitrary predicate tests. The syntax (var (fn arg0 arg1 ...)) applies the predicate function to the argument at runtime: (apply fn var arg0 arg1 ...).

1) *Basic Predicate Specialization*: Predicates work with any function, providing flexible dispatch:

```

;; Equality predicates
(defn check-value
  ([ (n (= 0))] :zero)
  ([ (n (= 1))] :one)
  ([n] :other))

;; Comparison predicates
(defn classify-number
  ([ (n (< 0))] :negative)
  ([ (n (= 0))] :zero)
  ([ (n (> 0))] :positive))

;; Range checking
(defn age-group
  ([ (age (< 13))] :child)
  ([ (age (< 20))] :teen)
  ([ (age (< 65))] :adult)
  ([age] :senior))

;; Symbol matching with quoted values
(defn dispatch-command
  ([ (cmd (= 'start))] (start-system))
  ([ (cmd (= 'stop))] (stop-system))
  ([ (cmd (= 'restart))] (restart-system))
  ([cmd] (unknown-command cmd)))

```

2) *Multiple Parameter Predicates*: Predicate specializers support multiple parameters with independent predicates:

```

(defn compare-signs
  ([ (a (< 0)) (b (> 0))] :negative-positive)
  ([ (a (> 0)) (b (< 0))] :positive-negative)
  ([ (a (= 0)) (b (= 0))] :both-zero)
  ([a b] :other))

(compare-signs -5 10) ; => :negative-positive
(compare-signs 10 -5) ; => :positive-negative

```

3) *Custom Predicates*: Any function can serve as a predicate, enabling domain-specific dispatch:

```

;; Type predicates
(defn process-value
  ([ (<number>? x)] (* x 2))
  ([ (<string>? x)] (str x x))
  ([ (<vector>? x)] (map process-value x))
  ([x] x))

;; User-defined predicates
(defn even? [n] (= (mod n 2) 0))
(defn odd? [n] (= (mod n 2) 1))

(defn classify-parity
  ([ (n (even?))] :even)
  ([ (n (odd?))] :odd))

```

4) *Predicate Specificity*: Predicate specializers have specificity level 4, making them more specific than type specializers (level 3), which are in turn more specific than destructuring patterns (level 1) and any patterns (level 0):

```

(defn check
  ([ (x (= 5))] :exactly-five)
  ; Predicate (level 4) - most specific
  ([ (x <number>)] :some-number)
  ; Type (level 3)
  ([x] :anything))
; Any (level 0) - least specific

(check 5) ; => :exactly-five
(check 10) ; => :some-number
(check "x") ; => :anything

```

5) *Integration with fn and lambda*: Predicate specializers work seamlessly in anonymous functions:

```

;; Inline predicate dispatch
((fn ([ (x (< 10))] :small)
      ([ (x (>= 10))] :large)) 5)
; => :small

;; Lambda with predicates
(def classifier
  (lambda ([ (n (< 0))] :neg)
        ([ (n (> 0))] :pos)
        ([n] :zero)))

```

6) *Restrictions and Design Rationale*: Predicate specializers are **not allowed** in macro parameter lists because macros receive unevaluated forms. A predicate needs to evaluate its argument, but macros work with raw syntax:

```

;; This signals an error:
(defmacro bad-macro
  ([ (x (= 0))] `0)
  ([x] `(inc ~x)))

; ERROR: Predicate specializers cannot be
;       used in DEFMACRO parameter lists
;       (macros receive unevaluated forms)

```

This restriction prevents confusion between compile-time pattern matching (on syntax) and runtime value testing (on evaluated data). The error is detected immediately during macro definition, providing clear feedback.

7) *Implementation*: Predicate specializers are compiled to a signature format (`:pred fn-name (arg0 arg1 ...)`) during function definition. At runtime, pattern matching evaluates `(apply fn-name actual-arg arg0 arg1 ...)` and dispatches to the clause when the result is truthy. The implementation maintains $O(N)$ dispatch time where N is the number of patterns at a given arity, with patterns sorted by specificity for efficient matching.

This design provides powerful dispatch capabilities while maintaining simplicity and integration with FOL's existing pattern matching infrastructure.

V. CLOJURE COMPATIBILITY

FOL maintains compatibility with key Clojure features to enable code reuse and familiar programming patterns:

Transducers: FOL implements Clojure's transducer protocol [6], enabling composable algorithmic transformations that work across different contexts (reduction, sequence building, channels). Example: `(transduce (comp (filter even?) (map square) (take 5)) + 0 (range))`.

Lazy Sequences: FOL provides lazy sequences via `lazy-seq`, enabling infinite data structures and efficient pipeline processing. The implementation uses thunks that cache results after first evaluation.

Sequence Operations: FOL provides Clojure's sequence abstraction with operations like `map`, `filter`, `reduce`, `take`, `drop`, and their lazy variants. These work uniformly across vectors, lists, sets, and dictionaries.

This compatibility ensures that FOL programmers familiar with Clojure can leverage existing knowledge while gaining the benefits of CLOS integration and predicate specializers.

VI. SELF-HOSTED EVALUATOR

FOL includes a meta-circular evaluator written in FOL itself (approximately 350 lines):

```
(defgeneric eval-form [form env])

(defmethod eval-form [(<number>? form) env]
  form) ; Self-evaluating

(defmethod eval-form [(<symbol>? form) env]
  (lookup env form))

(defmethod eval-form [(<list>? form) env]
  (if (empty? form)
      form
      (bind [op (first form)
             args (rest form)]
            (if (special-form? op)
                ((get special-forms (name op))
                 args env)
                (bind [fn-val (eval-form op env)
                       evald-args
                        (map #(eval-form % env) args)]
                      (apply-function fn-val evald-args)))))
  ))
```

The evaluator demonstrates several FOL features:

- Generic function dispatch on form types
- Pattern matching through method specialization
- First-class functions and lexical closures
- Special form handling through a dispatch table

Special form evaluators are defined as regular functions:

```
(defn eval-if [args env]
  (bind [arg-count (size args)]
        (if (or (< arg-count 2) (> arg-count 3))
            (throw "if: expected 2 or 3 args")
            (bind [test (nth args 0)
                   then-form (nth args 1)
                   else-form (if (>= arg-count 3)
                                (nth args 2)
                                nil)]
                  (if (fol-eval test env)
                      (fol-eval then-form env)
                      (fol-eval else-form env))))))
```

The self-hosted evaluator serves multiple purposes:

- 1) **Specification**: Defines FOL semantics precisely
- 2) **Metaprogramming**: Enables runtime code generation
- 3) **Extensibility**: Allows user-defined special forms

VII. EVALUATION

A. Performance Characteristics

FOL's performance characteristics reflect its implementation strategy:

TABLE II
PERFORMANCE CHARACTERISTICS

Operation	Persistent	Mutable
Vector access	$O(\log n)$	$O(1)$
Vector update	$O(\log n)$	$O(1)$
Dict lookup	$O(\log n)$	$O(1)$
Dict insert	$O(\log n)$	$O(1)$
Set membership	$O(\log n)$	$O(1)$

While persistent structures incur a logarithmic factor, the constant factors are small due to high branching factors (32-64). For typical applications, the difference is negligible compared to I/O and algorithmic complexity.

We have not performed comprehensive performance benchmarking against other Lisp implementations because FOL is currently interpreted while most Common Lisps and Clojure are compiled. Direct performance comparisons would be misleading, as the overhead of interpretation dominates execution time regardless of algorithmic choices. Once compilation is implemented (see Section VIII), meaningful performance comparisons will be possible. At that point, we expect FOL to perform comparably to Common Lisp for object-oriented code and to Clojure for functional operations, with the overhead of persistence being the primary differentiator.

B. Comparison with Related Languages

FOL uniquely combines persistent collections with full CLOS/MOP integration and extends pattern dispatch with

TABLE III
LANGUAGE FEATURE COMPARISON

Feature	FOL	Clojure	CL
Persistent collections	✓	✓	×
CLOS/MOP	✓	×	✓
Transducers	✓	✓	×
Lazy sequences	✓	✓	×
Pattern dispatch	✓	✓	×
Predicate specializers	✓	×	×
Macros	✓	✓	✓

general predicate specializers. This combination enables programming patterns unavailable in either Clojure or Common Lisp alone.

VIII. FUTURE WORK

Several extensions are planned for FOL:

A. Compilation

The most pressing need is compilation. Currently, all code is interpreted through the evaluator, which limits performance. We plan to implement compilation targeting Common Lisp as an intermediate representation, leveraging SBCL’s native code generation. This will enable:

- Substantial performance improvements (expected 10-100× speedup)
- Meaningful performance benchmarking against other Lisp implementations
- Production deployment for performance-critical applications
- Optimization of pattern dispatch and predicate specializers

B. Concurrency and Parallelism

FOL will adopt Clojure’s concurrency primitives to enable safe concurrent programming:

- **Atoms:** Synchronous, independent state with compare-and-swap semantics
- **Refs and STM:** Coordinated, synchronous state changes with software transactional memory
- **Agents:** Asynchronous, independent state changes
- **core.async:** CSP-style channels for asynchronous programming
- **Parallel collections:** Fork-join parallelism for sequence operations (pmap, fold, etc.)

These features leverage FOL’s persistent data structures, which are inherently thread-safe for reading.

C. Metadata System

We plan to add Clojure-style metadata to all FOL objects:

- Attach metadata dictionaries to any object without modifying the object itself
- Support for type hints, documentation, and user-defined annotations
- Integration with the MOP for metadata-driven behavior

D. Enhanced I/O and Streams

Current I/O capabilities are basic. Planned enhancements include:

- Additional stream classes for files, sockets, and in-memory buffers
- Standardized `*in*` and `*out*` dynamic variables for default streams
- Transducer-aware stream processing
- Integration with core.async channels for asynchronous I/O

E. Class System Extensions

- **Abstract classes:** Classes that cannot be instantiated directly
- **Sealed classes:** Classes that cannot be subclassed
- **Interfaces:** Pure protocol definitions without implementation

F. Error Handling Enhancements

While FOL currently supports Clojure-style `try/catch` error handling, we plan to enhance the system with:

- CLOS-style condition classes for structured error hierarchies
- Restarts and handlers for more sophisticated error recovery
- Integration with the MOP for customizable error behavior

G. Performance Optimization

Beyond compilation, planned optimizations include:

- Memory overhead reduction for object representation
- Optimized pattern dispatch compilation
- Specialized representations for small collections
- Transient collections for performance-critical mutation-heavy code (like Clojure)

IX. RELATED WORK

A. Clojure

Clojure [4] pioneered persistent data structures in production Lisp. FOL adopts Clojure’s sequence abstraction, transducer protocol and persistence while diverging in object system design. Where Clojure uses protocols and records, FOL uses CLOS generic functions and classes.

B. Common Lisp

FOL’s object system is a direct descendant of CLOS [2]. The meta-object protocol [3] enables deep customization of class and generic function behavior. FOL extends this with persistent slot values.

C. Dylan

Dylan [5] influenced FOL’s naming conventions and multiple dispatch semantics. The `<type>` notation improves code readability by clearly distinguishing types. In addition, FOL adopts Dylan’s module system. Dylan modules provide a simplified way to package symbols when compared with Clojure’s namespaces and Common Lisp’s packages.

D. Persistent Data Structures

Okasaki's work [7] on purely functional data structures underpins modern implementations. FSet [8] and Sycamore provide production-quality persistent collections for Common Lisp.

X. CONCLUSION

FOL demonstrates that functional and object-oriented programming are synergistic. By combining Clojure's persistent data structures with CLOS's powerful object system, FOL enables new programming patterns unavailable in either parent language.

The language's key contributions include:

- Integration of persistent collections and objects with CLOS
- Self-hosted evaluator showcasing metaprogramming
- Unified syntax spanning multiple Lisp traditions
- Production-ready implementation in Common Lisp

FOL shows that the Lisp family's evolution need not be divergent. By thoughtfully combining ideas from different dialects, we can create languages that are more than the sum of their parts.

The complete FOL implementation, including 6,721 tests and comprehensive documentation, is available at <https://github.com/frankadrian/fol>. The test suite comprises 6,144 bootstrap tests that exercise the interpreter's primitives and built-in operations, and 577 integration tests that validate FOL language functionality and semantics.

ACKNOWLEDGMENT

The author wishes to thank the Common Lisp and Clojure communities for their foundational work that made FOL possible.

REFERENCES

- [1] G. L. Steele Jr., *Common LISP: The Language*, 2nd ed. Digital Press, 1990.
- [2] D. G. Bobrow, L. G. DeMichiel, R. P. Gabriel, S. E. Keene, G. Kiczales, and D. A. Moon, "Common lisp object system specification," *ACM Sigplan Notices*, vol. 23, no. SI, pp. 1-142, 1988.
- [3] G. Kiczales, J. des Rivières, and D. G. Bobrow, *The Art of the Metaobject Protocol*. MIT Press, 1991.
- [4] R. Hickey, "The Clojure programming language," in *Proceedings of the 2008 Symposium on Dynamic Languages*, 2008.
- [5] Apple Computer, Inc., *Dylan Interim Reference Manual*. Apple Computer, Inc., 1993.
- [6] R. Hickey, "Transducers," *Clojure Blog*, August 2014. Available: <https://clojure.org/reference/transducers>
- [7] C. Okasaki, *Purely Functional Data Structures*. Cambridge University Press, 1999.
- [8] S. L. Siskind, "FSet: A functional set-theoretic collections library for Common Lisp," *Quicklisp*, 2012. Available: <https://common-lisp.net/project/fset/>